

PilotFish NULL Transport – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

This transport is a **deliberate no-op**. When a transaction reaches this stage, nothing is sent anywhere — the route still finishes successfully. Use it as a placeholder outbox, a testing sink, or the last stage when your “delivery” already happened earlier (for example in a processor).

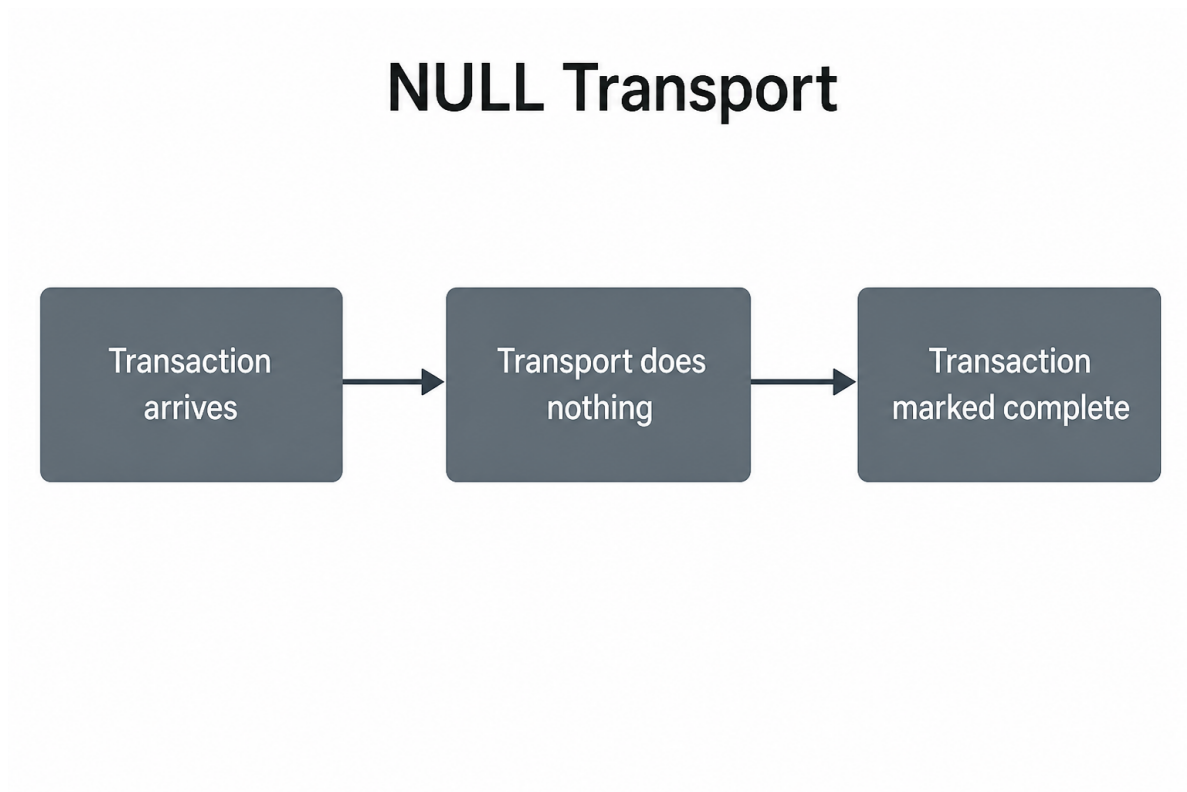


Figure 1: At a glance (plain English)

- **Does nothing** to the payload or remote systems.
- **Marks** the transaction complete via the standard transport wrapper.
- **Has no** configuration fields in the eiConsole UI.
- **Useful** for stubs, dry-runs, and routes that end after internal work.

Purpose

This document is a **deep dive** into the **NULL Transport** as shipped in `eip.war.hs.26R1.11`.

Provenance	Value
WAR	<code>eip.war.hs.26R1.11</code>
Module JAR	<code>xcs-modules-core-1.0.3.jar</code>
Decompiler	CFR 0.152
Implementation class	<code>com.pilotfish.eip.modules.other.NullTransport</code>
Decompiled path	<code>war-pipeline/decompiled/eip-26R1.11/.../NullTransport.java</code>
Transport type string	NULL Transport
Description	A NULL Transport that does nothing.
Help ID	<code>console.transport.null</code>
Category	PILOT_FISH

Derived from WAR bytecode (CFR). Narrative follows executable logic in `executeTransport`.

1. Conceptual model

```
TransactionData
|
v
AbstractTransport.sendData
|
+-- checkRequiredData
+-- executeTransport -> (empty body)
|
v
setTransactionStateComplete()
```

There is no network I/O, no file write, and no attribute mutation in this class.

2. High-level behavior (`executeTransport`)

1. `AbstractTransport.sendData` calls `checkRequiredData`, then `executeTransport`.
2. `NullTransport.executeTransport` is an empty method — returns immediately.
3. On success, base class calls `data.setTransactionStateComplete()`.
4. Any unexpected exception in the wrapper would set error state and wrap as `EIPEException("Error executing transport", e)` — this module itself throws nothing.

3. Configuration reference

3.1 Module-specific options

UI label	Tag	Type	Default	Tab	Required	Nuances
<i>(none)</i>	—	—	—	—	—	Descriptor title is "NULL"; no <code>PanelItem</code> entries

`getConfigurationDescriptor()` builds new `ConfigurationDescriptor(true)` and only sets the title. Completeness rules are therefore vacuously satisfied for module-local items.

3.2 Inherited `AbstractTransport` / `AbstractEIPModule`

Behavior	Detail
Module type	<code>ModuleType.TRANSPORT</code>
<code>sendData</code>	<code>checkRequiredData -> executeTransport -> complete / error</code>
<code>supportsBatchSynchronization()</code>	<code>false</code> (base default)
Parent <code>RoutingTarget</code>	Set via <code>setParent</code> / <code>getParent</code> (unused by this class)

4. Transaction attributes

4.1 Written by this transport

None.

4.2 Consumed

None.

5. Worked example — stub sink after side-effect processor

1. Route uses a Database SQL processor that already commits the business write.
2. Transport stage is **NULL Transport**.
3. Transaction ends complete without a second hop.

Useful when a transport stage is mandatory in the routing model but delivery is not.

6. Known quirks and caveats

1. **Payload preserved** — stream and attributes are untouched; downstream synchronous reply stages (if any) still see original data.
 2. **Not a discard-for-error** — errors earlier in the route still fail normally; NULL only skips *transport-side* delivery.
 3. **No logging inside execute** — operational visibility depends on surrounding route/transaction monitors.
-

7. Operational recommendations

Goal	Guidance
Unit-test a route without outbound connectivity	Use NULL until real transports are wired
Avoid double-send	Prefer NULL when an earlier stage already delivered
Document intent	Name the transport stage clearly (e.g. <code>NoOutbound-Stub</code>) so ops do not assume drops

8. Source index (this WAR)

Topic	Path under <code>war-pipeline/decompiled/eip-26R1.11/</code>
NULL Transport	<code>com/pilotfish/eip/modules/other/NullTransport.java</code>
Abstract transport base	<code>com/pilotfish/eip/extend/AbstractTransport.java</code>
Module JAR	<code>war-pipeline/jars/eip-26R1.11/xcs-modules-core-1.0.3.jar</code>

9. Pipeline provenance

PilotFish WARs/`eip.war.hs.26R1.11`

-> `xcs-modules-core-1.0.3.jar`

-> CFR decompile

-> `Documents/Transports/26R1.11/PilotFish-NULL-Transport-Reference-26R1.11.(md|pdf)`

Deep-dive documentation from WAR decompile – NULL Transport – 26R1.11 – August 2026.